

Drone Detection, Tracking and Trajectory Prediction

ML System Design Report

Domain: Counter-UAS Real-Time Perception **Scope:** Algorithm design, model selection, engineering justification, measured results

Abstract

A real-time machine learning pipeline that detects drones in video, tracks each across frames, and forecasts its position 1 to 2 seconds ahead, designed for edge deployment with a per-frame budget under 33 ms. The core contribution is a hybrid predictor that pairs a Kalman filter with an LSTM, keeping forecasts physically plausible while still following non-linear motion. A prototype runs the full chain at about 48 fps on a laptop, and the benchmark shows the learned predictor clearly beating physics-only baselines at longer horizons.

1. Problem and Objectives

Maintain a continuously updated picture of every drone in a long-range camera feed (up to about 1 km with suitable optics), each with a stable identity and a short trajectory forecast.

Why this is hard:

- **Tiny targets:** a drone may be under 10 pixels; a fixed-size resize erases it.
- **Low confidence:** distant drones score low, and naive trackers discard them.
- **Non-linear motion:** drones bank and accelerate; one fixed model cannot keep up.
- **Generalisation:** it must hold up across unfamiliar drone types, sizes, and environments.

Constraints: full pipeline under 33 ms (30 fps); recall before precision at detection (a missed target cannot be recovered); runs on an embedded GPU.

Approach: decompose into three independent stages, detection, tracking and prediction, each given the best-suited algorithm and connected through a clean interface, so any stage can be retrained or replaced without disturbing the others.

2. Algorithm Design

2.1 Detection

Choice: single-stage YOLOv8 / YOLO11. The detector runs every frame, so throughput sets the pipeline rate.

Approach	Speed	Small objects	Edge export
Two-stage (Faster R-CNN)	Slow on edge	Good	Awkward
Single-stage (YOLO)	Fast (30 to 120 fps)	Good with right head	ONNX / TensorRT
Transformer (DETR)	Moderate	Good	Moderate

Small-target measures: sliced inference (SAHI) keeps native resolution on tiles; a P2 head adds a dedicated band for the smallest objects; recall-biased NMS keeps uncertain boxes, since a missed detection cannot be recovered later.

Training: mosaic augmentation, scale jitter to 50%, and hard-negative mining on birds.

2.2 Tracking

Choice: ByteTrack. Two-pass association is the key feature.

- **Pass 1:** match high-confidence detections (score ≥ 0.25) using IoU plus the Kalman-predicted position (Hungarian assignment).
- **Pass 2:** match the remaining low-confidence boxes (down to 0.10) to unmatched tracks, recovering distant drones that other trackers drop.

Lifecycle: Tentative $\rightarrow$ Confirmed $\rightarrow$ Coasting $\rightarrow$ Deleted. The confirmation gate is the primary defence against false alarms; a one-frame artefact never accumulates enough matches to confirm. Coasting holds identity through brief occlusion.

Parameter	Value
High / low confidence threshold	0.25 / 0.10
New-track threshold	0.25
Track buffer (coasting)	30 frames
Match IoU threshold	0.80

2.3 Motion Features and Prediction: Kalman Filter and LSTM

Motion features. From each track's centre history the system derives position, velocity, acceleration and heading; this per-track state sequence is the input to both predictors below.

Why pair them. A Kalman filter is stable and never physically implausible but lags sharp manoeuvres; a network learns complex motion but can drift without an anchor. Run in parallel, the filter grounds the forecast and the network covers what physics cannot.

Kalman filter (physics). A 6D state (position, velocity, acceleration on both axes), measuring position only. Each frame it predicts forward with a constant-acceleration model, then corrects toward the new detection, trusting whichever is more certain. Rolling the predict step forward H times gives the forecast. Production upgrade: an IMM filter (constant-velocity, constant-acceleration, constant-turn-rate) that re-weights as motion changes.

LSTM (learned). A single layer, 64 hidden units, fed the last 10 frame-to-frame displacements and emitting the next 15, in displacement space so it is translation-invariant (Huber loss for robustness). It runs in two modes: direct, and a residual mode that learns a correction on top of a constant-velocity physics prior. The residual mode is more accurate at longer horizons (Section 3) and is the implemented default. The model also reports a per-step uncertainty envelope, which the demo draws around the forecast. Further upgrade: a temporal Transformer for multi-agent and longer-horizon forecasting.

2.4 Data, Training and Generalisation

Data assumptions. A fielded system assumes calibrated EO/IR cameras with time-synchronised frames and pose metadata; the prototype works from a single uncalibrated video in pixel coordinates.

Datasets. Detector: a Roboflow drone set (about 4900 / 1400 / 700 train / val / test images, two classes). Predictor: trajectories harvested from tracked video plus a synthetic generator (straight, curved and accelerating paths).

Training strategy. Transfer-learn the detector from COCO weights with small-object augmentation (mosaic, scale jitter, flips) and hard-negative mining on birds. Train the LSTM on whole-trajectory splits, so no track appears in both train and test and there is no leakage.

Generalisation across types, sizes, environments. *Types:* recall-first, near class-agnostic detection plus the per-track classifier, so an unfamiliar airframe is still found. *Sizes:* scale augmentation, the P2 head and tiling cover a few pixels up to a close pass, and the displacement-space predictor is size-invariant. *Environments:* training-data diversity, EO/IR fusion for low visibility, and scenario-based validation that holds out whole conditions rather than random frames.

2.5 Production-Grade Pseudocode

The production pipeline is written as a streaming algorithm. Each frame is processed once, while the system maintains persistent state for every active track.

Algorithm 1: Production Drone Detect-Track-Predict Pipeline

Inputs:

```
video_stream or camera_stream
detector_weights                // fine-tuned YOLO model
tracker_config                 // ByteTrack thresholds and buffers
lstm_checkpoint, optional
H = 15                         // prediction horizon in frames
HIGH_CONF = 0.25
LOW_CONF = 0.10
CONFIRM_FRAMES = 5
TRACK_BUFFER = 30
```

Persistent state:

```
active_tracks = empty set
history[track_id] = queue of recent centre points
kalman[track_id] = constant-acceleration Kalman filter
class_votes[track_id] = class vote counter
area_history[track_id] = recent bounding-box areas
alerted_tracks = empty set
```

Initialisation:

```
load YOLO detector on edge GPU
load ByteTrack tracker
if lstm_checkpoint exists and compute budget allows:
    load residual LSTM predictor
else:
    use Kalman-only degraded mode
```

For each incoming frame F_t :

1. Acquire and preprocess frame
 - timestamp F_t
 - resize or tile frame according to deployment mode
 - normalise image for detector
2. Detect candidate drones
 - detections = YOLO(F_t)
 - detections = recall_biased_NMS(detections)
 - high_detections = boxes with confidence $\geq$ HIGH_CONF
 - low_detections = boxes with LOW_CONF $\leq$ confidence $<$ HIGH_CONF
3. Predict current track locations before association
 - for each track in active_tracks:
 - predicted_position = kalman[track.id].predict_one_step()
 - adapt search gate using estimated speed and covariance
4. Associate detections to tracks using ByteTrack
 - matches_1, unmatched_tracks, unmatched_high = HungarianMatch(active_tracks, high_detections, cost = IoU + KalmanGate)
 - matches_2, unmatched_tracks, unmatched_low = HungarianMatch(unmatched_tracks, low_detections, cost = IoU + KalmanGate)
 - for each unmatched high-confidence detection:
 - create tentative track with new track_id
5. Update track state
 - for each matched track and detection:
 - centre = centre_of(detection.box)
 - smoothed_centre = kalman[track.id].update(centre)
 - append smoothed_centre to history[track.id]
 - update class_votes[track.id] using detection.class
 - update area_history[track.id] using detection.box_area
 - mark track as matched in this frame
 - for each unmatched active track:
 - keep Kalman prediction as temporary position
 - increase missed-frame counter
 - if missed-frame counter $>$ TRACK_BUFFER:
 - delete track
 - for each tentative track:
 - if matched for CONFIRM_FRAMES:
 - promote to confirmed track
 - if not matched consistently:
 - delete as false alarm
6. Classify behaviour and raise alerts
 - for each confirmed track:
 - speed = kalman[track.id].estimated_speed()
 - area_trend = trend(area_history[track.id])
 - if speed is near zero for many frames:

```

        behaviour = HOVER or LOITER
    else if area_trend is increasing:
        behaviour = APPROACH
    else if area_trend is decreasing:
        behaviour = DEPART
    else:
        behaviour = TRANSIT

    if track.id not in alerted_tracks:
        emit confirmed-drone alert
        add track.id to alerted_tracks

7. Predict future path
    for each confirmed track:
        kalman_path = kalman[track.id].predict_future(H)

        if LSTM is available and history[track.id] has enough points:
            displacement_window = last K frame-to-frame displacements
            residual_path, uncertainty = LSTM(displacement_window)
            lstm_path = constant_velocity_prior(history[track.id]) + residual_path
            final_path = lstm_path
        else:
            final_path = kalman_path
            uncertainty = covariance_from_Kalman()

8. Publish outputs
    draw or transmit:
        bounding box, stable track ID, class label, behaviour label
        Kalman forecast, LSTM forecast, uncertainty envelope
    log latency, FPS, alerts and track summaries

9. Real-time safety check
    if frame latency exceeds budget or GPU is saturated:
        reduce detector image size or skip LSTM
        continue Kalman-only tracking instead of stopping the stream

```

Outputs:

- confirmed drone tracks with stable IDs
- current position, speed and behaviour per track
- 1-2 second trajectory forecast with uncertainty
- alert events for newly confirmed drones

3. Results

Latency (Apple M3, MPS, YOLOv8n at 640 px, 432 frames):

Stage	Time
Capture + preprocess	~1.0 ms
YOLO detection	~6.4 ms
ByteTrack + Kalman	~1.0 ms
LSTM prediction	~1.0 ms
Draw + encode	~11.0 ms
End to end	~20.9 ms (47.9 fps)

The 33 ms budget leaves about 12 ms of headroom for production additions.

Prediction benchmark (synthetic, 1000 / 900 / 800 windows; pixels, lower is better):

Predictor	H=5 ADE	H=10 ADE	H=15 ADE	H=15 FDE	H=15 MR@20
Constant-velocity	3.27	6.18	9.84	21.07	23.2%
Kalman (CA)	4.68	10.90	20.34	50.22	87.2%
LSTM (direct)	1.93	3.36	5.34	11.37	12.0%
LSTM (residual)	1.92	3.22	5.15	10.83	11.1%

Both LSTM variants win at every horizon. The residual variant, which learns a correction on top of a physics prior, edges out the direct one and by a widening margin at longer horizons, which is why it is the default. The

constant-acceleration Kalman trails the constant-velocity baseline here because the synthetic paths oscillate and the CA model overshoots; on smoother real footage the filter is far more competitive, which is exactly what the hybrid design covers.

Detection baseline: stock YOLOv8n scores near zero ($\text{mAP}@.5 = 0.008$) because "drone" is not a COCO class. The fine-tuning script is provided; this is the expected starting point, reported honestly.

4. Edge Cases

- **Fast drones:** the velocity estimate sizes the search gate, so a fast target is not lost.
- **Sudden manoeuvres:** tune process noise; the LSTM detects the early shape; the IMM upgrade handles model switching structurally.
- **False alarms:** the confirmation gate, hard-negative training, and track coasting filter them in layers.
- **Compute degradation:** if the GPU saturates, the Kalman filter runs alone; the pipeline degrades gracefully rather than stalling.

5. Production Extensions

The demo now includes a basic version of classification and behaviour; the rest are design-only (detailed as diagrams in the architecture document):

- **Classification** (basic version in demo): the demo votes the detector's class per track for a stable label; the production version is a dedicated airframe-type classifier (quad, hex, fixed-wing, VTOL) with an open-set unknown class.
- **Behaviour / intent** (basic version in demo): the demo tags hover, loiter, approach, depart and transit by geometric rules; production adds a learned anomaly model.
- **Multi-camera fusion:** track-level fusion across separated cameras; triangulation gives 3D position.
- **World coordinates:** pixel to geodetic (lat, lon, alt, speed, heading) with uncertainty; the largest gap to deployment.
- **Transformer prediction:** the upgrade path for swarms and longer horizons, with the filter as anchor.

6. Limitations

- Works in image pixels, not metric coordinates (needs calibration and altitude).
- Single EO camera; a fielded system fuses EO, IR and radar/RF.
- Tracking metrics (MOTA, IDF1, HOTA) need MOT-format ground truth the dataset lacks.
- No sliced inference in the prototype; whole-frame detection is used instead.

Conclusion

Three decisions define the design: single-stage YOLO for edge throughput, ByteTrack for its low-confidence recovery of distant drones, and a Kalman-plus-LSTM predictor that is both physically grounded and able to follow manoeuvres. The prototype runs the full chain at about 48 fps and halves the physics-only prediction error at the longest horizon. The gaps to a fielded system, sensor fusion, world coordinates and tracking ground truth, are stated explicitly and mapped to a clear production path.